

Search Typeahead System

Project Report — High-Level Design Assignment

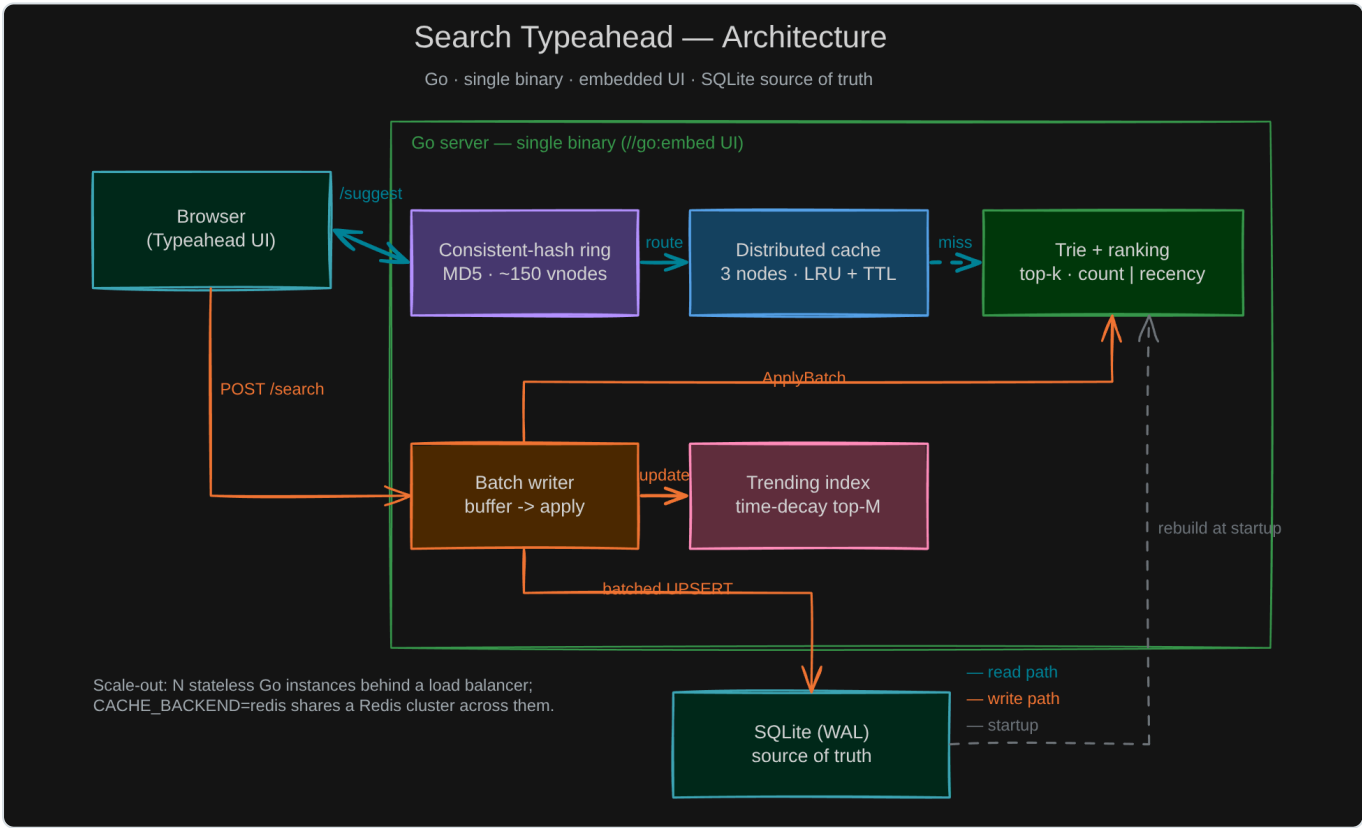
Author: Parth Sankhla · Date: 22 June 2026 · Stack: Go (single binary) + embedded HTML/JS UI + SQLite

This is a typeahead (search autocomplete) service. As the user types, it returns matching suggestions, ranked either by all-time popularity or by what people have searched recently, and it keeps a short list of trending queries. When a search is submitted it gets written back asynchronously, so the read path stays fast. The whole thing runs as one Go binary: the HTTP API, the in-memory index, the cache, and the web UI (compiled in with `//go:embed`). The only external dependency is a pure-Go SQLite driver. This report covers the architecture, the dataset and how to load it, the API, the design decisions I made and their trade-offs, and the performance I measured.

1. Architecture · 2. Dataset & loading · 3. API documentation · 4. Design choices & trade-offs · 5.
- Performance report

1. Architecture

Requests come in over HTTP. Reads (`/suggest` , `/trending`) are served from an in-process distributed cache that sits in front of an in-memory Trie. Writes (`POST /search`) are recorded asynchronously and folded into both the index and the database. SQLite holds the durable data, and everything in memory is rebuilt from it when the server starts.



Read path (cyan): browser to the consistent-hash ring, to a cache node, and on a miss to the Trie. Write path (orange): `POST /search` to the batch writer, then on to the Trie, the trending index, and SQLite. The dashed line is the Trie and trending index being rebuilt from SQLite at startup.

Read path

The prefix is first normalised (trimmed and lower-cased). The **consistent-hash ring** then maps it to one of N logical **cache nodes**. If that node already has the prefix, its candidate set is re-ranked for the requested mode and returned. If it does not, the **Trie** produces the candidates: its precomputed top-k for that prefix, plus any trending queries that also match the prefix. The result goes into the cache, then gets ranked and returned. On the common path nothing touches SQLite.

Write path

A submitted search returns straight away and is handed to an in-memory **batch writer**. Events go onto a buffered channel; a drain goroutine collects them, adds up duplicate queries, and once a batch is big enough or enough time has passed, applies it to the Trie's counts and the trending index and writes it to SQLite in a single transaction. A lot of searches end up as a handful of database writes.

Startup

When the server boots it reads every row out of SQLite to rebuild the Trie and seed the trending index. SQLite is the one source of truth, and the process keeps no durable state of its own apart from the cache.

Components

Component	Responsibility
HTTP API + embedded UI	The endpoints and the single-page UI, served from one binary via <code>//go:embed</code> .
Consistent-hash ring	Maps a prefix key to the cache node that owns it (MD5, ~150 virtual nodes per node).
Distributed cache	N logical LRU + TTL nodes behind the ring; in-process by default, Redis-backed if you switch it on.
Trie	Prefix index with precomputed per-node top-k candidates; rebuilt from SQLite at startup.
Ranking	Scores candidates by <i>count</i> or <i>recency</i> (time-decayed), per request.
Trending index	A bounded top-M of recently-searched queries by time-decayed activity.
Batch writer	Buffers, aggregates and flushes search events to the Trie and SQLite.
SQLite store	The durable source of truth (WAL mode); read from at startup, written to in batches at runtime.
Metrics	A lock-free latency histogram (p50/p95/p99) and atomic cache/DB counters.

2. Dataset source and loading

Committed sample (works out of the box)

The repo includes `backend/data/queries.csv`, a **100,000-query synthetic dataset** built by `cmd/gen` with heavy-head (Zipf-like) counts, so the project clears the assignment's $\geq 100\text{k}$ -row minimum without downloading anything. Each line is `query,count`.

Real open-source dataset (English Wikipedia pageviews)

For a real corpus, `scripts/fetch_dataset.sh` pulls one hour of [Wikimedia pageview dumps](#) (a public dataset). The most-viewed article titles become queries, weighted by their view counts. I validated the system on the top **117,182** titles.

Loading

```
# 1) Synthetic – regenerate at 150k, load into SQLite, run
cd backend
make seed      # cmd/gen → data/queries.csv (150k), then load → SQLite
make run       # builds the Trie from SQLite and serves on :8080

# Or reload whatever CSV is committed
make load      # cmd/load -data data/queries.csv -db data/typeahead.db

# Without make
go run ./cmd/load -data data/queries.csv -db data/typeahead.db
go run ./cmd/server -db data/typeahead.db

# 2) Real Wikipedia dataset (top 150k titles by views)
./scripts/fetch_dataset.sh 20240101-120000 data/wiki_full.csv
sort -t, -k2,2 -nr data/wiki_full.csv | head -150000 > data/queries.csv
make clean load run
```

`cmd/load` reads the CSV in one transaction, normalising queries and summing duplicate counts. Loading 150k rows takes about 0.9 s, and the Trie builds in under a second.

3. API documentation

Base URL (local): `http://localhost:8080`. All responses are JSON. Query strings are normalised (trimmed and lower-cased) on the server, so `IPhone`, `iphone` and `iphone` all behave the same.

GET /suggest — prefix suggestions

Param	Req	Default	Notes
<code>q</code>	no	—	the typed prefix; empty or missing returns trending
<code>mode</code>	no	server default (<code>recency</code>)	<code>count</code> (all-time) or <code>recency</code> (popularity plus recent activity)

```
GET /suggest?q=ip&mode=count
200 OK
[
  {"query":"iphone","score":100000},
  {"query":"iphone 15","score":85000},
  {"query":"ipad","score":70000}
]
```

`score` is whatever the chosen mode ranked by: the raw count in `count` mode, or the combined $\log(1+\text{count}) + \alpha \cdot \text{recency}$ in `recency` mode. No matches returns `200 []`. An empty `q` returns the trending list (same shape).

POST /search — record a search

Records a submitted search and returns a dummy response. The query is buffered and written in a batch, so its effect on `/suggest` and `/trending` shows up after the next flush.

```
POST /search {"q":"iphone"}
200 OK       {"message":"Searched"}
400 Bad Request {"error":"empty query"}
```

GET /trending — top queries by recent activity

```
GET /trending
200 OK
[
  {"query":"javascript","score":50.0},
  {"query":"java rocks","score":32.0}
]
```

`score` is the effective (decayed-to-now) recency value. It's empty until some searches have been recorded.

GET /cache/debug — inspect ring routing and cache state

Shows how a prefix routes on the consistent-hash ring and whether it's currently cached. It doesn't mutate anything and is left out of the hit/miss metrics. `prefix` is required.

```
GET /cache/debug?prefix=ip
200 OK
{ "prefix":"ip", "key":"ip", "hash":10771293625506255592,
  "node":"cache-1", "vnode_pos":10772485985601381633, "state":"hit" }
```

`state` is one of `hit`, `miss` or `expired`. A `/suggest` for a cold prefix flips a later probe from `miss` to `hit`.

GET /metrics — operational counters

```
GET /metrics
200 OK
{
  "latency": { "cached": { "count":367847, "p50_us":25, "p95_us":25, "p99_us":50},
               "cold":   { "count":477, "p50_us":25, "p95_us":100, "p99_us":400} },
  "cache":   { "hits":367847, "misses":477, "hit_rate":0.9987,
               "nodes":{"cache-0":179, "cache-1":141, "cache-2":153}},
  "writes":  { "received":6000, "dropped":0, "upserts":144, "flushes":12, "reduction_factor":41.67},
  "db":       { "reads":150000, "writes":144}
}
```

Status codes & static UI

`200` for success (including empty arrays); `400` for an empty query or prefix on `POST /search` or `/cache/debug`; `404` for an unknown path with no static asset. `GET /` (and `/app.js`, `/style.css`) serve the embedded single-page UI, with no build step or separate server.

4. Design choices and trade-offs

One Go binary with the UI embedded STACK

The API, index, cache and the static UI all run in one process, and the UI is compiled in with `//go:embed .`

Why: there's one thing to build and run (`go build` , then run it), no Node toolchain, and no CORS to deal with because the UI is same-origin. It's also easier to reason about the system as a whole, which helps when I have to explain it. **Trade-off:** the frontend and backend are versioned and deployed together. For a read-mostly service that scales by running more copies behind a load balancer, that coupling hasn't cost me anything.

SQLite through a pure-Go driver, in WAL mode STORAGE

Durability is handled by `modernc.org/sqlite` (no cgo) with write-ahead logging.

Why: there's no external service to install, and with a cgo-free driver the binary builds statically (`CGO_ENABLED=0`). WAL mode lets the startup reader and the batch writer work without blocking each other. **Trade-off:** it's a single-node embedded database, not a networked one. That's fine here because SQLite is only read at startup and written to in batches, never on the read path.

A Trie with precomputed per-node top-k INDEX

A prefix trie where every node already holds its top-k descendants by count, so a `/suggest` is a short walk to the prefix node and a read of its list.

Why: lookup is $O(\text{prefix length})$ with no per-request sorting of subtrees, which is what keeps cache-miss latency under a millisecond. The lists are built once with a post-order DFS after the bulk load; at runtime, a changed query bubbles up through its ancestors' lists. **Trade-off:** it uses more memory and does more work on writes than a plain trie. That's a deliberate choice to make reads cheap, which is what matters for typeahead.

Consistent hashing with MD5 and virtual nodes CACHING

Keys land on a 64-bit ring via MD5, with about 150 virtual nodes per logical cache node and a deterministic tie-break.

Why: adding or removing a cache node moves only about $1/(N+1)$ of the keys instead of almost all of them, which is what `hash % N` would do, so scaling doesn't wipe the cache. The virtual nodes keep the load even. I picked MD5 after measuring the alternatives: **FNV-1a** clustered the sequential virtual-node ids and gave a $1.76\times$ load skew, and **maphash** seeds itself randomly per process, so placement isn't stable across restarts. **Trade-off:** MD5 is slower than FNV, but it only runs once per request on a short key, so it doesn't show up in the numbers, and in return the ring is even and reproducible.

A distributed cache of logical nodes that stores candidate sets CACHING

The cache is N independent LRU + TTL nodes behind the ring (in-process by default; set `CACHE_BACKEND=redis` to run them as real Redis servers with the same ring and routing). Each entry holds the prefix's *candidate set*, not a finished ranked list.

Why: caching candidates rather than a ranked result means one entry can serve both `count` and `recency` requests, and the re-rank on a hit runs over at most ~50 candidates, which is cheap. The Redis switch shows the same design running as a genuinely distributed cache without changing any application logic. **Trade-off:** a small re-rank on every hit, in return for a higher effective hit rate and a single code path for both ranking modes.

Recency ranking by exponential time-decay RANKING

Each query carries a decaying recency value, and the same `/suggest` serves both modes through `?mode=`. In recency mode the score is $\log(1+\text{count}) + \alpha \cdot \text{effective_recent}(\text{now})$, where `effective_recent` decays with half-life H ($\lambda = \ln 2/H$).

Why: the `log` compresses the heavy head, so a query that's genuinely trending can overtake one that's popular all-time but cold right now. Decay is computed from the current time on read, so reads never change state and ranking stays cheap and idempotent. **Trade-off:** there are two knobs to set (half-life and α), but they're plain config and make the difference between the two modes easy to demonstrate.

Trending as a bounded, self-evicting index RANKING

Trending is a capped top-M index (cap 2000) keyed by a *time-invariant* eviction key $s = \ln(\text{value}) + \lambda \cdot t_s$.

Why: because the key is time-invariant, I don't have to re-decay every entry on each update, so keeping the set is cheap and memory stays bounded no matter how big the vocabulary gets. **Trade-off:** the cap could in principle drop a borderline entry, but 2000 is well above what the trending head needs.

An asynchronous batch writer WRITES

Searches go to a buffered channel, then a drain goroutine, then an apply worker. A batch flushes when it reaches ≥ 500 aggregated queries or every 5 s. `Submit` is non-blocking but will block up to 50 ms before dropping under heavy backpressure. Persistence writes *absolute* counts, not deltas.

Why: aggregating turns many searches for the same query into one UPSERT (I measured $41.7\times$ fewer writes), which keeps the disk off the read path. Writing absolute counts rather than deltas means a failed flush needs no fix-up: the next flush just rewrites the correct value, and a clean shutdown flushes whatever is still buffered. **Trade-off:** a hard crash can lose the small, bounded window of un-flushed writes. That's acceptable for counts that get rebuilt from SQLite on the next start, and the bound is explicit (buffer size plus batch size).

Lock-free metrics OBSERVABILITY

Latency goes into an atomic histogram (p50/p95/p99, split by cache hit vs miss), and the cache and DB counters are atomics.

Why: measuring shouldn't slow down the thing being measured, so the counters stay off the contended path while still giving the numbers the assignment asks for at `/metrics`. **Trade-off:** fixed histogram buckets instead of exact quantiles, which is the usual cheap way to report latency online.

5. Performance report

All numbers here are measured from the Go implementation. Environment: Go 1.26, single binary, in-process cache (3 logical nodes $\times$ 150 vnodes), SQLite (modernnc.org/sqlite). Primary dataset: 150,000 distinct queries with heavy-head Zipf-like counts (`cmd/gen`). Load generator: `cmd/bench`, 50 concurrent workers, prefixes drawn from the dataset.

5.1 Latency & throughput

Two views here: **client-side** end-to-end (HTTP plus Go client contention under 50 workers on localhost) and **server-side** (the suggestion API's own work, from `/metrics`).

Mode	Throughput	Client p50	Client p95	Client p99	Server p50 (cached)	Server p95 (cached)
count (basic)	30,546 req/s	392 μ s	6.29 ms	17.8 ms	25 μ s	25 μ s
recency (enhanced)	30,319 req/s	404 μ s	7.02 ms	18.3 ms	25 μ s	25 μ s

The API serves a cached prefix in about 25 μ s on the server side. The few-millisecond client p95 is HTTP round-trip plus `net/http` client and GC contention at 30k req/s on a single machine, not the index itself. Recency mode barely costs anything, since the re-rank is over at most $\sim$ 50 candidates. A server-side cache miss is still sub-millisecond (p50 25 μ s, p95 100 μ s, p99 400 μ s): a Trie walk, the candidate union, and a re-rank.

5.2 Cache hit rate (consistent-hash distributed cache)

From `/metrics` after the count run: a **0.9987 hit rate** (367,847 hits against 477 misses), with per-node entries of 179 / 141 / 153. The set of hot prefixes stays cached, so the only misses are first-touch.

5.3 Write reduction (batching)

6,000 `POST /search` across 12 distinct queries, then `/metrics`: `received 6000` `·` `upserts 144` `·` `flushes 12` `·` `dropped 0` `·` `41.7×`. The 6,000 searches became 144 database UPSERTs with nothing dropped. The ratio rises as traffic concentrates, which is what a typeahead workload does.

5.4 Database read/write counts

`db.reads = 150,000` is the one-time scan that builds the Trie at startup; `db.writes = 144` are the runtime batched UPSERTs. The read path does no database reads at all.

5.5 Consistent hashing (`cmd/rebalance`, 100k keys)

Scenario	Result
Distribution over 3 nodes (MD5, 150 vnodes)	33.1% / 34.5% / 32.4% ($\approx$ 1.04 $\times$ skew)
Add a 4th node — keys moved	24.9% ($\approx$ 1/(N+1))

Baseline <code>hash % N (3 → 4)</code> — keys moved	75.1%
---	-------

Adding a node moves about 25% of the keys versus about 75% for plain `hash % N`, roughly a 3× cut in cache churn, and the distribution stays even. That's the reason for picking MD5 over FNV-1a, which measured a 1.76× skew.

5.6 Ranking: basic vs enhanced (same `/suggest`, prefix `ja`)

After `java` and `java tutorial` were each searched 500× (a write burst):

Rank	mode=count (all-time)	mode=recency (recency-aware)
1	jacket salad dessert — 19657	java tutorial — 995
2	java jacket javascript — 8067	java — 995
3	java toaster mercedes — 7705	jacket salad dessert — 9.9

Count mode ranks by all-time popularity. Recency mode pushes the freshly-searched `java` and `java tutorial` to the top, while `jacket salad dessert`, popular but cold, drops down. It's the same endpoint; `?mode=` switches the behaviour.

5.7 Real Wikipedia dataset (cross-check)

Re-running on real English Wikipedia pageviews (top 117,182 titles by views for one hour) gives consistent results:

Metric	Synthetic 150k	Wikipedia 117k
Throughput	30,546 req/s	27,120 req/s
Server cached p50 / p95	25 / 25 µs	25 / 50 µs
Cold (miss) p95 / p99	100 / 400 µs	100 / 400 µs
Cache hit rate	99.87%	94.4%
Client p50 / p95	392 µs / 6.3 ms	628 µs / 6.6 ms

The lower hit rate on Wikipedia is expected. 117k diverse real titles make a larger, more spread-out working set (around 6k entries per node versus ~150 on the head-heavy synthetic set), so there are more first-touch misses. It still holds above 94%, server-side latency stays around 25–50 µs, and the keys spread evenly (6819 / 5602 / 5782).

5.8 Supporting evidence


```
shanks@hld-assignment:~/backend$ go run ./cmd/rebalance
```

```
=== Consistent hashing demo: 100000 keys ===
```

```
Distribution over 3 nodes (MD5, 150 vnodes):
```

node-0	33059	(33.1%)
node-1	34549	(34.5%)
node-2	32392	(32.4%)

```
After adding node-3 – keys moved: 24924 (24.9%)
```

node-0	22343	(22.3%)
node-1	27364	(27.4%)
node-2	25369	(25.4%)
node-3	24924	(24.9%)

```
Baseline `hash % N` (3→4 nodes) – keys moved: 75083 (75.1%)
```

```
Consistent hashing moves  $\sim 1/(N+1)$  of keys; `hash % N` moves  $\sim 75\%$ .
```

`cmd/rebalance` : an even split across 3 nodes, with 24.9% of keys moving when a 4th node is added versus 75.1% for `hash % N`.

```
shanks@hld-assignment:~/backend$ curl -s localhost:8080/metrics | python3 -m json.tool
```

```
{
  "cache": {
    "hits": 189769,
    "misses": 483,
    "hit_rate": 0.9974612619052625,
    "nodes": {
      "cache-0": 180,
      "cache-1": 142,
      "cache-2": 153
    }
  },
  "db": {
    "reads": 100000,
    "writes": 6
  },
  "latency": {
    "cached": {
      "count": 189769,
      "avg_us": 19,
      "p50_us": 25,
      "p95_us": 50,
      "p99_us": 100
    },
    "cold": {
      "count": 483,
      "avg_us": 95,
      "p50_us": 100,
      "p95_us": 200,
      "p99_us": 3200
    }
  },
  "writes": {
    "received": 16,
    "dropped": 0,
    "upserts": 6,
    "flushes": 5,
    "reduction_factor": 2.6666666666666665
  }
}
```

GET /metrics : latency split into cached and cold, the cache hit rate with per-node entry counts, and the batch-write counters.

SEARCH TYPEAHEAD

ja	SEARCH
ja va youtube coffee	142,953
ja cket drums	52,696
ja vascript case protein	17,722
ja cket protein	12,204
ja vascript pixel	11,796
ja cket thermos sale	7,574
ja cket project	7,246
ja vascript tripod desktop	7,190
ja vascript case coffee	5,894
ja cket haskell	5,844

The embedded typeahead UI. Suggestions update as you type, with the matched prefix in bold and the score on the right.

To reproduce the performance numbers: `go run ./cmd/gen -n 150000 -out data/bench.csv`, then `go run ./cmd/load ...`, run the server, then `go run ./cmd/bench -c 50 -d 10s` and `go run ./cmd/rebalance`. The code is laid out as `backend/cmd/` (entrypoints), `backend/internal/` (one package per component) and `backend/web/` (the embedded UI).